Assignment 3

1. Create a Flask application with an /api route. When this route is accessed, it should return a JSON list. The data should be stored in a backend file, read from it, and sent as a response.

App.py

Data.json

Explanation:

This Flask application provides an /api route that returns data in JSON format. The data is stored in a backend file (such as a data.json file). When the /api endpoint is accessed, the application reads the data from the file, converts it into a Python object, and sends it as a JSON response using Flask's jsonify function. This demonstrates how to build a simple API, read data from a file, and return it to the client in JSON format.

2. Create a form on the frontend that, when submitted, inserts data into MongoDB Atlas. Upon successful submission, the user should be redirected to another page displaying the message **"Data submitted successfully"**. If there's an error during submission, display the error on the same page without redirection.

App.py

app.py > ...

```python
from flask import Flask, render_template, request, redirect, url_for
from pymongo import MongoClient

app = Flask(__name__)

# 🔗 Replace with your MongoDB Atlas connection string
MONGO_URI = "mongodb+srv://koushiksmenon5_db_user:Kailasam123@cluster0.1cjfzhs.mongodb.net/?appName=Cluster0"

client = MongoClient(MONGO_URI)
db = client["mydatabase"]
collection = db["users"]

@app.route('/')
def form():
    return render_template('form.html', error=None)

@app.route('/submit', methods=['POST'])
def submit():
    try:
        name = request.form.get('name')
        email = request.form.get('email')

        if not name or not email:
            return render_template('form.html', error="All fields are required")

        # Insert into MongoDB
        collection.insert_one({
            "name": name,
            "email": email
        })

        return redirect(url_for('success'))

    except Exception as e:
        return render_template('form.html', error=str(e))

@app.route('/success')
def success():
    return render_template('success.html')

if __name__ == '__main__':
    app.run(debug=True, use_reloader=False)
```

Form.html

templates > form.html > ...

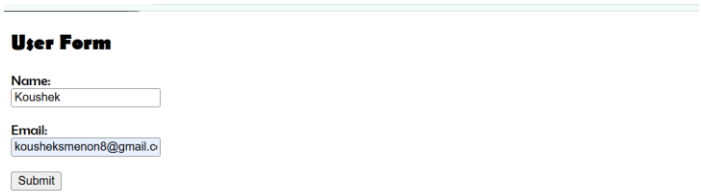
```html
<!DOCTYPE html>
<html>
<head>
    <title>Form</title>
</head>
<body>
    <h2>User Form</h2>

    {% if error %}
        <p style="color:red;">{{ error }}</p>
    {% endif %}

    <form action="/submit" method="POST">
        <label>Name:</label><br>
        <input type="text" name="name"><br><br>

        <label>Email:</label><br>
        <input type="email" name="email"><br><br>

        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

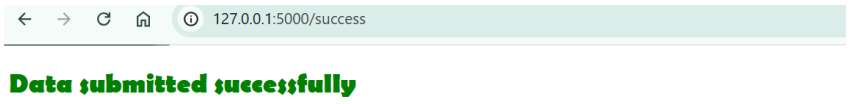
Success.html

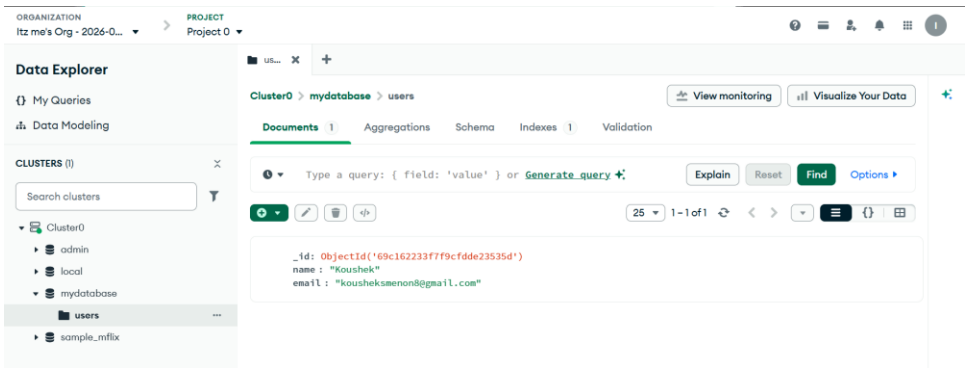
templates > success.html > ...

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Success</title>
5  </head>
6  <body>
7      <h2 style="color: green;">Data submitted successfully</h2>
8  </body>
9  </html>
```

Solution







Explanation:

This project is a Flask-based web application where users submit a form from the frontend. The entered data is sent to the backend and stored in MongoDB Atlas. If the submission is successful, the user is redirected to a success page showing "Data submitted successfully". If any error occurs, it is displayed on the same page without redirection. The application demonstrates basic form handling, database integration, and error management.